Scan for Artifact

SPECULATE: Generating REST API Specifications Using LLMs

Krishanu Singh [1] Kushagra Karar [1] Abhilash Jindal [1] (Advisor) Guowei Yang [2] (Co-Advisor)

[1]Indian Institute of Technology (IIT) Delhi [2]University of Queensland, Australia

Research Track: Track 1: Core AI/ML & Trustworthy Systems

REST API Specifications

OpenAPI Specification (OAS):

```
paths:
  /api/products/:
    get:
      summary: Retrieve
        products.
      parameters:
        - name: q
          in: query
          schema:
            type: string
      responses:
        '200':
          description:
            Success
          content:
            application/json:
              schema:
                type:
                  object
                properties:
                  id:
                    type:
                      integer
```

- OAS makes it easier to write client libraries, automated tests etc.
- Manually writing OAS is a massive developer burden.
- Automated tools have critical limitations too.
  - Dynamic Analysis (e.g., ExpressO): Captures live API traffic, but misses unexecuted paths.
  - Static Analysis (e.g., Respector): Inspects code via framework rules but is tightly bound to the specific frameworks. DRF Spectacular – Django, Respector – Jersey, Spring.

Limitations of Static Analysis Tools: Template Deviations

Mozilla Treeherder (Django):

```
def list(self, request):
    # Developer bypasses DRF templates
    page = request.query_params.get('page')
    limit = request.query_params.get('limit')

    queryset = self.get_queryset()
    return Response(queryset[page:limit])
```

DRF Spectacular vs. Ground Truth:

```
   parameters:
-    # SOTA solvers miss these entirely!
+    - name: page
+      in: query
+    - name: limit
+      in: query
```

Static solvers fail when developers bypass standard templates.

Limitations of Static Analysis Tools: Path Explosions

Senzing API (Jersey):

```
@GET @Path("entity-networks")
public SzResponse getEntityNetwork(
    @QueryParam("e") List<String> eParam,
    @QueryParam("entities") String eList) {

    if ((eParam == null || eParam.isEmpty()) &&
        (eList == null || eList.isEmpty())) {
        throw new BadRequestException("...");
    }
}
```

Respector vs Ground Truth:

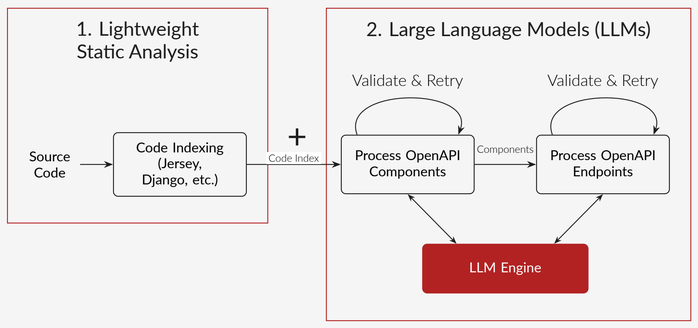
```
paths:
  /entity-networks:
    get:
      parameters:
        - name: entities
          in: query
-         required: true
+         required: false
```

Respector's strict path pruning leads to incorrect specifications

Core Hypothesis

- Use static analysis only to **index** the code, not to interpret it.
- Let an **LLM reason** over the code index.
- Spec generation then no longer depends on framework templates or path pruning, so it **generalizes across languages and frameworks**.

Architecture Overview



SPECULATE combines lightweight static analysis with LLM's reasoning power to generalize across languages and frameworks.

Phase 1: Lightweight Static Analysis

Source Code (Python):

```
# catalog/serializers.py
class PartnerSyncRequestSerializer(
    serializers.Serializer
):
    partner = serializers.ChoiceField(
        choices=partner_choices()
    )

    dry_run = serializers.BooleanField(
        required=False, default=True
    )

    since = serializers.DateTimeField(
        required=False
    )
```

AST Parsed Global Index:

```
{
  ".../catalog/serializers.py": {
    "classes": {
      "ProductSerializer": { "..." },

      // --- EXTRACTED AST NODE ---
      "PartnerSyncRequestSerializer": {
        "startLine": 72, "endLine": 75,
        "properties": ["partner", "dry_run", "since"],
        "functions": ["partner_choices"]
      },

      "PartnerSyncResponseSerializer": { "..." }
    },
    "functions": {}, "identifiers": { "..." }
  }
}
```

Phase 2: LLM Generation & Validation

LLM Prompt (Endpoint Resolution):

```
Task: Generate OpenAPI 3.0 YAML for POST /api/products/partner-sync/
Constraint: Leverage exact $ref linking from context.

Pre-Computed Context ($ref Registry):
- PartnerSyncRequestSerializerRequest
- PartnerSyncResponseSerializerResponse

Target Source Code:
class ProductPartnerSyncView(APIView):
    def post(self, request):
        # ... imperative payload logic ...
```

Generated Specification:

```
/api/products/partner-sync/:
  post:
    requestBody:
      content:
        application/json:
          schema:
            $ref: >-
              #/components/schemas/
              PartnerSyncRequestSerializerRequest
    responses:
      '202':
        content:
          application/json:
            schema:
              $ref: >-
                #/components/schemas/
                PartnerSyncResponseSerializerResponse
```

Validation Loop: Checked for Structural, Referential, & OpenAPI Integrity (Max 3 Retries)

Algorithmic Novelty

- **Dependency-ordered generation:** Components are topologically sorted and generated before endpoints, so every `$ref` resolves.
- **Targeted context:** The index returns only the code the LLM needs (parents, field types, concrete classes).

Evaluation Setup & Metrics

Dataset: 19 Real-World Repositories

Framework	Language	# Repos
Django	Python	4
Jersey	Java	8
Spring Boot	Java	7
Total	-	19

SPECULATE is evaluated on a diverse dataset:

- **1.0 to 118.8 KLoC** per repository (406.6 KLoC in total)
- **5 to 278 endpoints** per repository (1315 endpoints in total)
- **Diverse domains:** CI, databases, traffic analytics, public procurement, workload automation, etc.; **repositories 4–16 years old**, with **up to 9.4K GitHub stars**

SOTA Baselines:

- **DRF-Spectacular** (Django)
- **Respector** (Java - Jersey/Spring)
- **DevSpec** (Java - Spring)

5 Evaluation Metrics (Manual Ground Truth):

1. Endpoint Methods 2. Req. Params 3. Req. Constraints 4. Res. Params 5. Res. Constraints

```
paths:
  /api/products/resolve/:
    get:
      parameters:
        - name: external_id
          in: query
          schema:
            type: string
            pattern: ^ext_[0-9]
      responses:
        '200':
          content:
            application/json:
              schema:
                properties:
                  related:
                    type: array
```

RQ1: Can SPECULATE Generate Accurate Specifications?

Framework	Tool	Endpoint (P / R)	Req Params (P / R)	Req Const (P / R)	Resp Params (P / R)	Resp Const (P / R)
Django	SPECULATE	100 / 100	92 / 95	85 / 84	88 / 82	78 / 76
(Python)	DRF-Spectacular	98 / 88	50 / 57	32 / 35	67 / 52	53 / 52
Java	SPECULATE	100 / 98	96 / 92	90 / 89	94 / 84	85 / 82
(Jersey & Spring)	Respector	93 / 98	85 / 47	81 / 42	61 / 14	61 / 12
	Dev Spec	93 / 84	80 / 65	77 / 59	54 / 52	49 / 45

SPECULATE outperforms SOTA on deep response schemas and complex constraints.

RQ2: Reasoning vs Non-reasoning LLMs' impact on SPECULATE

